Operating Systems Orange Problem

Name: Mahathi S

SRN:PES2UG24AM084

Class: B (AIML)

1.A

```
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9
db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227e
d0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$
```

1.B

```
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e
566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9
aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11e
c4f476
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$
```

2.A

```
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2.B

```
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a18
3d337639025de576db9ebb4
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$
```

3A.

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ ./pes status
Staged changes:
  staged:     file1.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  Makefile
  untracked:  pes.c
  untracked:  index.c
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  index.h
  untracked:  test_tree.c
  untracked:  README.md
  untracked:  pes.h
  untracked:  tree.h
  untracked:  object.c
  untracked:  commit.h

vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$
```

3B

```
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776406766 6 file1.txt
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$
```

4A

```
committed: f1487bc0e464... Add farewell
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$
./pes log
commit f1487bc0e46402feeebb33b3051df6b856a7eb3777a782ff0a68dd493ed3b35d
Author: Mahathi S
Date:   0



commit 299625d646604915f2e1e83b128e1d937799c565928bf78c4e45f1324e902e1a
Author: Mahathi S
Date:   0

    ◆◆

commit b897c0d9a489a83eaa3d97cded36e28fe08bf4c8cdcb01082c94780869f6ca5e
Author: Mahathi S
Date:   0



vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ find .pes -type f | sort
```

4b

```
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/29/9625d646604915f2e1e83b128e1d937799c565928bf78c4e45f1324e902e1a
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/b8/97c0d9a489a83eaa3d97cded36e28fe08bf4c8cdcb01082c94780869f6ca5e
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/487bc0e46402feeebb33b3051df6b856a7eb3777a782ff0a68dd493ed3b35d
.pes/refs/heads/main
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$
```

4c

```
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ cat .pes/refs/heads/main
f1487bc0e46402feeebb33b3051df6b856a7eb3777a782ff0a68dd493ed3b35d
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ cat .pes/HEAD
ref: refs/heads/main
vboxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$
```

Final:

```
/boxuser@UbuntuVM:~/PES2UG14AM084-pes-vsc$ make test-integration
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

```
--- First Commit ---
Committed: a1204bf5e82d... Initial commit

Log after first commit:
No commits yet.

--- Second Commit ---
Committed: 4a200fb71301... Update file.txt

--- Third Commit ---
Committed: 60252a15935d... Add farewell

--- Full History ---
commit 60252a15935d2d7110d82d8cac4b499ca67c920f8b7e979f79865c81e13b0cfe
Author: Mahathi S
Date:   0



commit 4a200fb71301ab3da278ea2f62e947d0c66583c28a7c11dadfa3f83c57c4a046
Author: Mahathi S
Date:   0



No commits yet.

--- Reference Chain ---
HEAD:
ref: refs/heads/main
```

```
HEAD:
ref: refs/heads/main
refs/heads/main:
60252a15935d2d7110d82d8cac4b499ca67c920f8b7e979f79865c81e13b0cfe

--- Object Store ---
Objects created:
8
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/4a/200fb71301ab3da278ea2f62e947d0c66583c28a7c11dadfa3f83c57c4a046
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/60/252a15935d2d7110d82d8cac4b499ca67c920f8b7e979f79865c81e13b0cfe
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/a1/204bf5e82da684aa3fba18a80ff53bb5acbdda275dc90338f088f184bf78f5
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Q5.1: How would you implement pes checkout <branch>?

A branch in PES-VCS is represented as a file inside .pes/refs/heads/ that stores the hash of the latest commit. The HEAD file points to the current branch.

To implement pes checkout <branch>, the following steps are required:

First, verify whether the specified branch exists by checking for the file .pes/refs/heads/<branch>. If it does not exist, the operation should fail with an error.

Update the HEAD file so that it points to the new branch. This is done by writing:ref: refs/heads/<branch>

Read the commit hash stored in the branch file and load the corresponding commit object from the object store.

From the commit object, retrieve the tree hash representing the snapshot of the project.

Reconstruct the working directory by traversing the tree object recursively. For each entry:

If it is a blob, recreate the file with its contents.

If it is a tree, recreate the directory and process its contents.

This operation is complex because it involves safely modifying the working directory. It must correctly handle file creation, deletion, and overwriting, while ensuring that no important data is lost. Additionally, it must consider uncommitted changes, which can make the operation unsafe if not handled properly.

Q5.2: How to detect a dirty working directory?

A “dirty working directory” refers to a situation where the user has made changes that are not yet committed.

To detect this, the system compares the working directory with the index:

The index stores metadata for each staged file, including file size, modification time, and hash.For each file in the index:

Compare the current file’s size and modification time with the stored values.

If they differ, recompute the file’s hash and compare it with the stored hash.

If the hash differs, the file has been modified and is considered “dirty.”

During checkout, if a file is modified in the working directory and also differs between the current branch and the target branch, a conflict occurs. In such cases, the checkout operation must be aborted to prevent loss of uncommitted changes.

Thus, detecting a dirty working directory relies on comparing:

The index (expected state)

The working directory (current state)

The object store (stored content)

Q5.3: Detached HEAD

A detached HEAD occurs when the HEAD file points directly to a commit hash instead of a branch reference.

In this state:

New commits can still be created.

However, these commits are not associated with any branch.

As a result, the new commits are not reachable through any branch and can eventually be lost during garbage collection.

To recover such commits, the user can:

Create a new branch pointing to the commit hash.

Or manually update a branch reference to include the commit.

Thus, while detached HEAD allows temporary exploration of past commits, it is risky because any new work may become unreachable if not explicitly saved.

Q6: Garbage Collection and Space Reclamation

Q6.1: Finding and deleting unreachable objects

Over time, the object store may contain objects that are no longer referenced by any branch. These are called unreachable objects.

To identify and remove them, a mark-and-sweep algorithm can be used:

Start from all branch heads located in .pes/refs/heads/.

For each branch:

Load the commit object.

Traverse its parent commits recursively.

For each commit, follow its tree.

For each tree, visit all blobs and subtrees.

Maintain a set of all visited (reachable) object hashes.

After traversal, scan the entire .pes/objects/ directory.

Any object not present in the reachable set is considered unreachable and can be safely deleted.

A hash set is the most suitable data structure for tracking reachable objects, as it allows constant-time lookup and efficient handling of large numbers of objects.

For a repository with 100,000 commits and multiple branches, the traversal may involve visiting hundreds of thousands of objects, including commits, trees, and blobs.

Q6.2: Why garbage collection is dangerous during a commit

Garbage collection is unsafe when executed concurrently with a commit operation due to possible race conditions.

A typical race condition scenario is:

A commit operation creates new objects (blobs, trees, commit) but has not yet updated the branch reference.

At the same time, garbage collection runs and scans for reachable objects.

Since the new commit is not yet referenced by any branch, GC considers its objects unreachable and deletes them.

The commit operation then completes and updates the branch to point to the now-deleted objects.

This results in a corrupted repository, where references point to non-existent data.

To avoid this, real systems like Git use several techniques:

Locking mechanisms to prevent concurrent modifications.

Atomic updates to ensure references are updated safely.

Temporary protection of newly created objects.

Delayed garbage collection, ensuring that only old unreachable objects are removed.

Thus, garbage collection must be carefully coordinated with commit operations to maintain repository integrity.